

Brain Agriculture API

Architecture & Domain Design

Backend technical challenge - NestJS, TypeORM, PostgreSQL and OpenTelemetry

Source of truth: project archive final.zip supplied for review.

1. Executive summary

Brain Agriculture is a REST backend for managing rural producers, farms, crop seasons, crops and plantings. The implementation uses NestJS and TypeScript, persists domain data in PostgreSQL through TypeORM, exposes OpenAPI documentation, and includes unit and end-to-end tests. Business dashboard endpoints aggregate farm, hectare, state, crop and land-use information.

The domain model keeps the planting of a crop separate from the crop catalog. A Planting associates one Crop with one CropSeason and stores planted area, which allows the same crop to be reused across farms and seasons without duplicating the crop definition.

2. Technology stack

Concern	Implementation
Runtime / language	Node.js 24, TypeScript, ES modules
Framework	NestJS 12
Persistence	PostgreSQL 18 + TypeORM
Validation	class-validator, custom CPF/CNPJ and decimal-string decorators
API contract	REST + Swagger / OpenAPI
Testing	Vitest, Supertest, unit tests and E2E tests
Packaging	Docker + Docker Compose
Observability	OpenTelemetry Node SDK, OTel Collector, Prometheus, Loki, Tempo and Grafana

3. C4 - System context

The primary actor is an API consumer such as a client, partner or another developer. The Brain Agriculture API owns the rural-producer domain and persists it in PostgreSQL. Operational telemetry is emitted through OpenTelemetry and is intended to be explored through the observability platform.

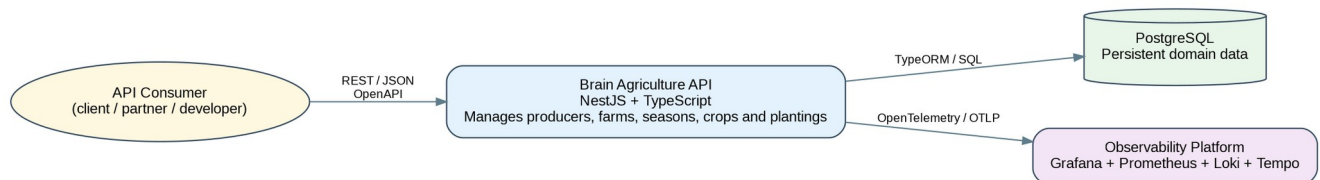


Figure 1 - C4 system context

4. C4 - Containers

At container level, the NestJS API is the application boundary. PostgreSQL is the system of record. The Docker Compose topology also includes pgAdmin and an observability stack composed of OpenTelemetry Collector, Prometheus, Loki, Tempo and Grafana.

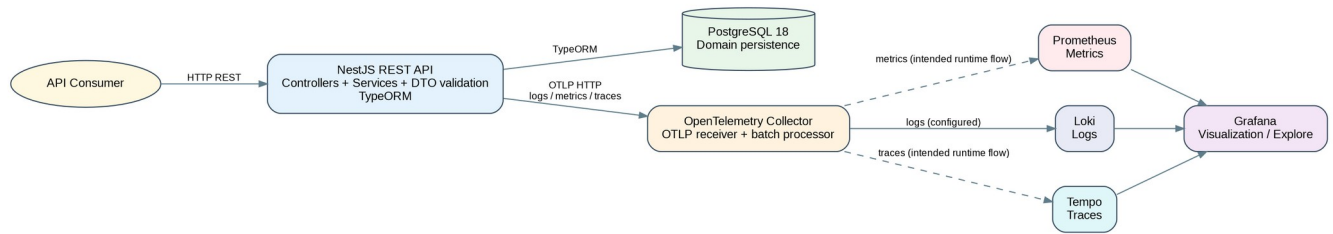


Figure 2 - C4 container view

5. Internal application architecture

The code follows a conventional NestJS layered structure. Controllers expose HTTP resources, DTOs validate the API contract, services implement domain rules and transactional consistency, and TypeORM repositories/DataSource provide persistence. Database migrations duplicate critical integrity rules where appropriate.

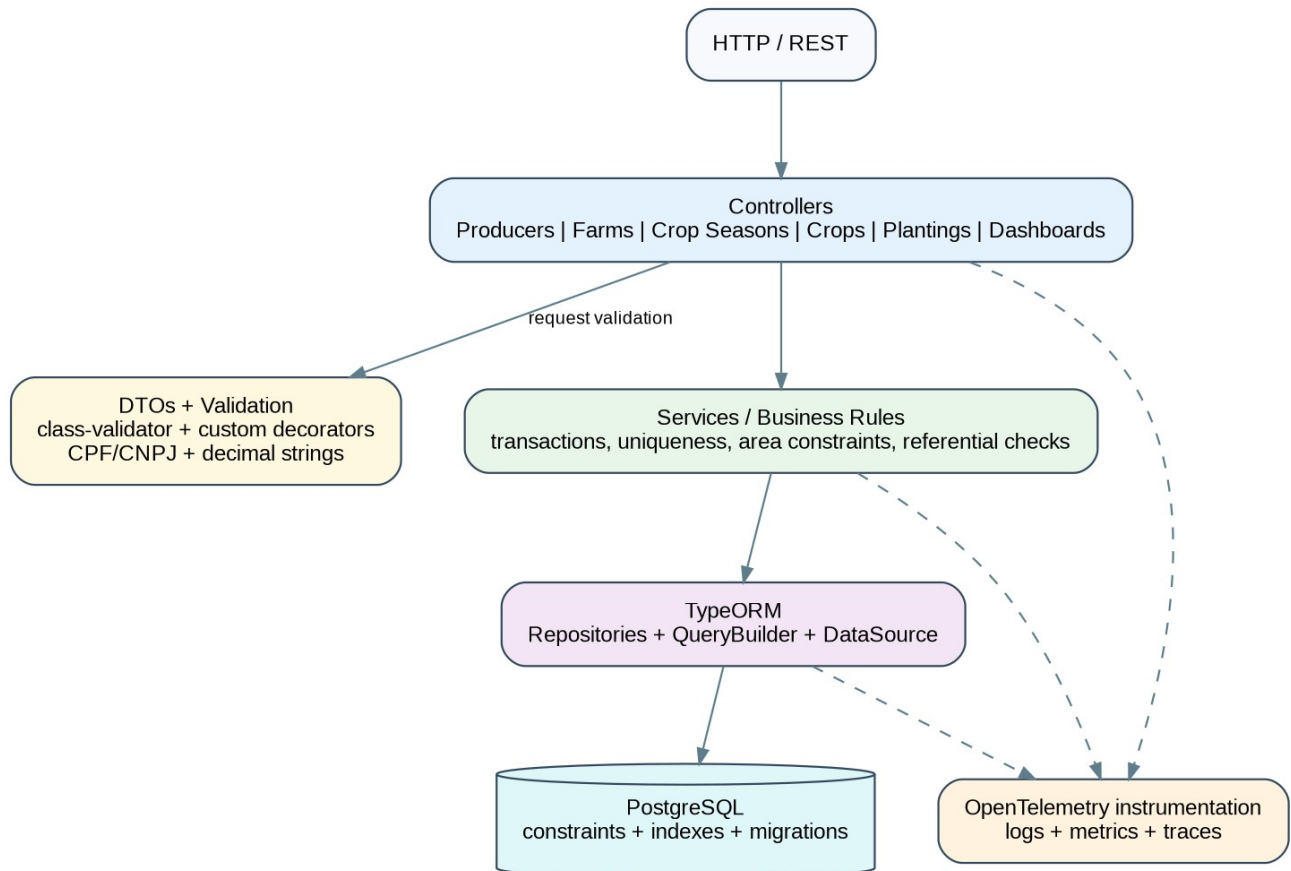


Figure 3 - Internal backend architecture

6. Domain and database model

The central relationship chain is Producer -> Farm -> CropSeason -> Planting. Crop is a reusable catalog entity referenced by Planting. UUIDs are used as primary keys. Decimal hectare values are stored as NUMERIC/DECIMAL(15,4) and represented as strings in the TypeScript entities/DTOs to avoid binary floating-point precision loss.



Figure 4 - Entity relationship diagram

6.1 Referential and uniqueness constraints

- `Producer.document` is unique.
- `Farm.producer_id` references `Producer`; deleting a producer with farms is restricted.
- `CropSeason` is unique by (`farm_id`, `year`); deleting a farm cascades to its crop seasons.
- Crop name is unique case-insensitively through a unique index on `LOWER(name)`.
- `Planting` is unique by (`crop_season_id`, `crop_id`), preventing the same crop from being registered twice in one season.
- Deleting a crop used by plantings is restricted; deleting a crop season cascades to its plantings.
- Farm migrations enforce positive total area, non-negative arable/vegetation areas and `arable + vegetation <= total`.
- Planting migrations enforce `planted_area_ha > 0`.

7. Business rules

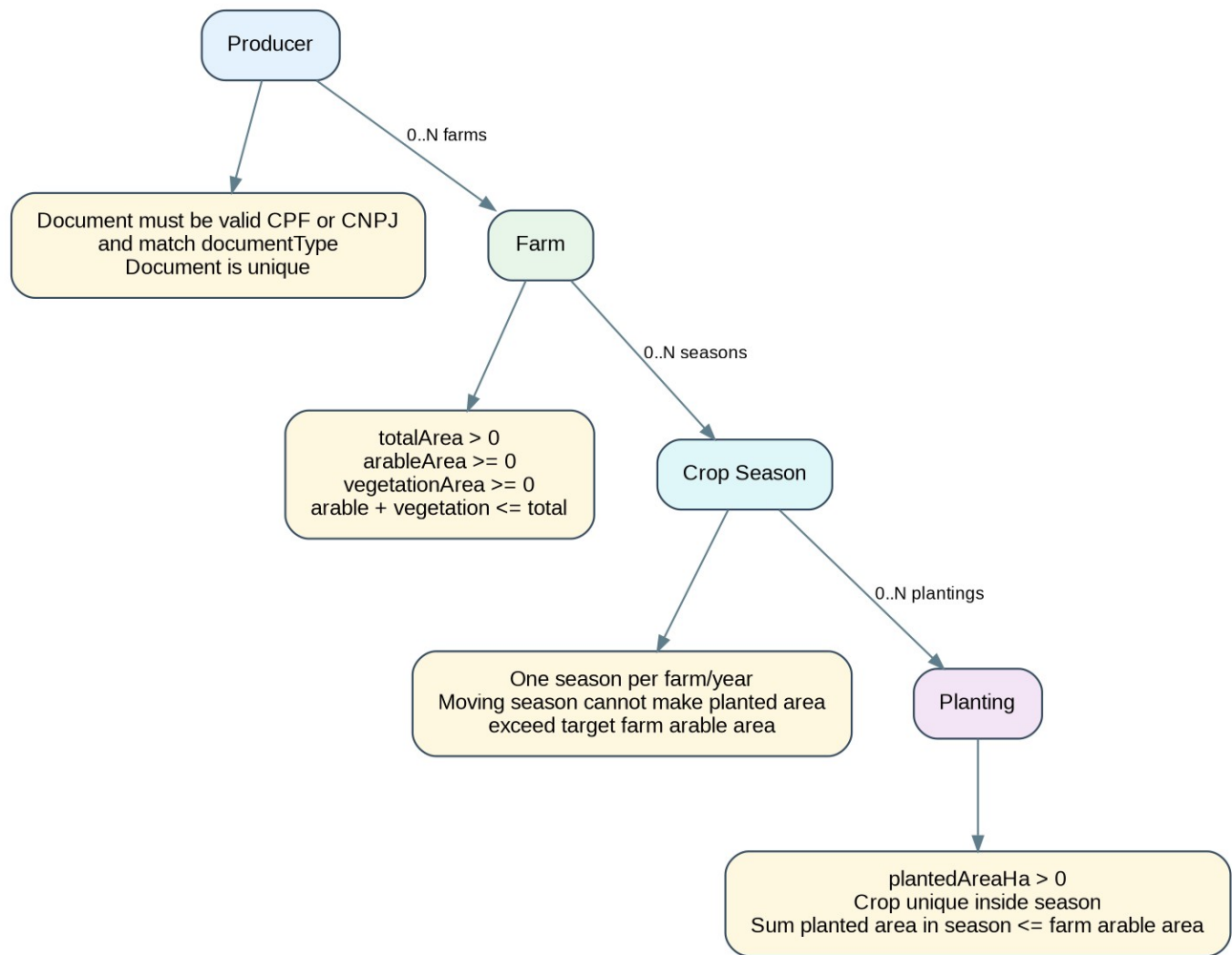


Figure 5 - Main business rules and invariants

- A producer has a valid CPF or CNPJ, and the document must be consistent with documentType.
- A producer may have zero or more farms.
- A farm must belong to an existing producer.
- Farm total area must be greater than zero; arable and vegetation areas cannot be negative.
- Arable area plus vegetation area cannot exceed total farm area.
- When reducing a farm's arable area, existing season plantings must still fit inside the new arable area.
- A farm may have zero or more crop seasons, but only one crop season for the same farm/year pair.
- A crop season may contain zero or more plantings.
- The same crop cannot be registered more than once in the same crop season.
- The sum of planted areas in a crop season cannot exceed the farm's arable area.
- Moving a crop season to another farm is rejected if its existing plantings exceed the target farm's arable area.

8. Business dashboard API

Because this submission is backend-only, the dashboard requirement is fulfilled by REST endpoints that return datasets ready for a frontend to render as cards and pie charts. Grafana is operational observability and is not the business dashboard.

Endpoint	Dataset
GET /dashboards/totals	COUNT farms and SUM total farm hectares
GET /dashboards/by-state	Farm count grouped by state
GET /dashboards/by-crop	Planting count and planted hectares grouped by crop
GET /dashboards/land-use	SUM arable hectares and SUM vegetation hectares

9. Observability architecture

The application initializes the OpenTelemetry Node SDK before the NestJS application, with automatic Node instrumentation and OTLP/HTTP exporters for traces, metrics and logs. The log pipeline carries trace_id and span_id, allowing log records to be correlated with request traces.

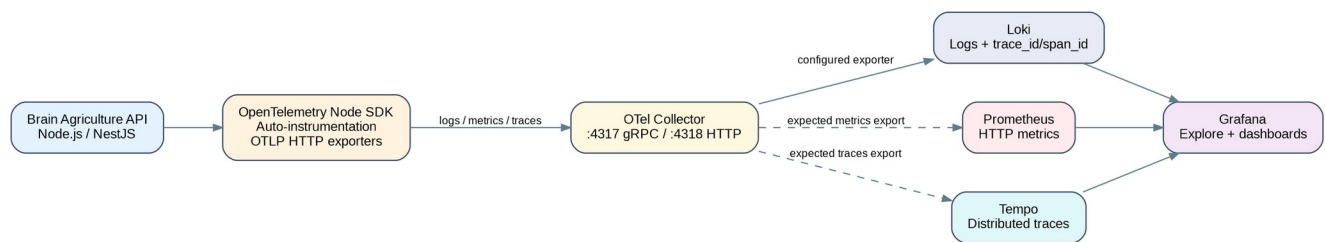


Figure 6 - Observability topology

9.1 Important implementation note

Archive configuration gap: in the supplied final.zip, observability/otel-collector/otel-collector.yml exports logs to Loki, but the traces and metrics pipelines currently export only to the Collector debug exporter. The Docker Compose file provisions Tempo, Prometheus and Grafana, and the application sends all three signal types to the Collector, but the archived Collector YAML does not contain the Tempo and Prometheus exporters required for those two signals. If the runtime environment was already tested successfully with Tempo/Prometheus, ensure that the working Collector configuration is copied into the repository before submission.

10. Testing strategy present in the repository

The repository contains unit tests for controllers, services, decorators and decimal utilities, plus E2E suites for producers, farms, crop seasons, crops, plantings and dashboards. Reusable factories, fixtures and TypeORM mocks are also present under test/.

- Unit test command: npm run test
- E2E test command: npm run test:e2e
- Coverage command: npm run test:cov

- Build command: `npm run build`
- Lint command: `npm run lint`

11. Architectural decisions

Decision	Rationale
Decimal areas as strings	Avoids loss of precision when exchanging PostgreSQL NUMERIC values with JavaScript.
Planting association entity	Models crop + season + planted area explicitly and supports many seasons/farms per crop.
Service + DB validation	Critical invariants are protected in application logic and, for key numeric/uniqueness rules, in PostgreSQL.
RESTRICT vs CASCADE deletes	Protects referenced producers/crops while allowing dependent seasons/plantings to follow their owning aggregate.
Separate business dashboard	Domain aggregations stay in the API; Grafana remains dedicated to technical observability.
OpenTelemetry	Keeps telemetry vendor-neutral and allows logs, metrics and traces to share service/resource context.

12. Submission checklist

- Replace the default NestJS README with project-specific setup, architecture, API, migrations, tests and observability instructions.
- Run build, unit tests, E2E tests and lint from a clean environment.
- Verify Docker Compose starts from documented environment variables.
- Ensure the repository contains the Collector configuration actually used for Prometheus and Tempo.
- Keep Swagger/OpenAPI reachable and document its route in README.
- Include this architecture document (or its PDF) under docs/.